Lab 16: Modules

- Due Thursday by 11:59pm
- Points 5
- Submitting an external tool
- Available Jul 7 at 6pm - Jul 9 at 11:59pm

Learning Objectives

- Practice writing tests for function.

Completion Requirements

- Submit q1.py, q2.py and q3.py.

Q1: isPrime and isBinary

In q1.py, create the following two functions.

- **isPrime(n):** The function returns true if n is a prime number, and false otherwise. **n** will be greater than 1. You can check if a number is prime by testing that it is not divisible by any number less than **n** except 1.
- **isBinary(n)**: The function returns true if the given string **n** only has 0's and 1's in it. i.e. the string represents a binary number.

Example:

```
>> isPrime(20)
False
>> isPrime(23)
True
>> isBinary("23")
False
>> isBinary("10101010100011101011")
True
```

Q2: Use q1 module

In q2.py, import the q1 module to use the **isPrime** and **isBinary**. Prompt the user to enter a binary number. Print "Invalid input" if the string is not a binary number. If the string is valid then convert the string to a base 10 integer and check if it is prime. Print if the number is prime or not.

Example 1:

```
Enter a binary number : 101
101 is Prime.
```

Example 2:

```
Enter a binary number : 100
100 is not Prime.
```

Q3: Use third party module

A module **wrongImplementations** is given to you which contains implementations of the following functions.

- **fibonacci (n)**
- **combinations (n, k)**

You need to import the wrongImplementations module and write a function **testing (n, k)** which computes the following expression:

(fibonacci(n) * combinations (n, k)) / fibonacci(fibonacci(n))

Note:
The implementations given to you are wrong. You will not get credit for writing and using your correct implementations. The credit is for using the module you don't know anything about. You can not test this locally.

This tool needs to be loaded in a new browser window

Load Lab 16: Modules in a new window